



IITISoC Problem Statements

Science & Technology Council, IIT Indore

Artificial Intelligence & Machine Learning

Index

Beginner

Student Document Assistant	4
Inventory Planning Assistant	6

Intermediate

Graphitti: Graph-Native Web Intelligence	9
SPECTRUM	13
VeriTrace AI: Multimodal Reasoning Auditor	17
Autonomous Financial Due Diligence Platform	20
Intelligent Cargo Packing & Spatial Neuro-Optimization	24

Advanced

RL-Based Personalized Chess Assistant	30
SPARK: Accelerated Image Generation	34
Personalized Mental Health Digital Twin using AI	38
Pathway-Powered Agentic Gateway	41
Building Lightweight Multi-Task AI with MALORA	46

Beginner



IITISoC Problem Statement

Science & Technology Council, IIT Indore

Student Document Assistant

Domain: AI / ML
Difficulty: Beginner
Team Size: 2–4 members

1. Background

Students often struggle to navigate long policy documents, FAQs, notices, and process guides for academic or administrative tasks. They usually need concise answers and relevant references, but the source material is spread across multiple documents and is tedious to read end-to-end. This problem focuses on building a practical GenAI assistant over a bounded document collection. The scope is still approachable for beginner teams, but the final product should do retrieval and grounded answer generation well, with clear citations and a usable interface.

2. Problem Statement

Core Objective: Design and implement a retrieval-augmented GenAI assistant for a focused domain such as student policies, hostel rules, admissions FAQs, placement guidelines, or scholarship information. Given a user query, the system should retrieve the right source material and generate a grounded answer with citations.

3. Scope of Work

3.1 Knowledge Base and Preprocessing

- Build a bounded knowledge base from PDFs, webpages, handbook sections, circulars, forms, or FAQ documents relevant to one focused domain.
- Clean, chunk, normalize, and document the ingestion workflow.
- Organize the source material so that answers can be traced back to specific documents or sections.

3.2 Core GenAI Pipeline

- Build a retrieval-backed generation pipeline using keyword search, embeddings, reranking, or other lightweight retrieval techniques.

- Generate grounded answers and concise summaries strictly from the retrieved source material.
- Include at least one trust-building mechanism such as source citations, retrieved context display, or answer-confidence heuristics.

3.3 Interactive Product Surface

- Provide a simple interface where a user can ask questions, inspect cited sources, and receive grounded answers.
- Allow users to compare retrieved passages with the final generated answer.
- Teams may optionally add query suggestions, multi-turn follow-ups, or document upload for the chosen domain.

4. Technical Constraints

- The full workflow must be reproducible on modest hardware or clearly document any cloud usage.
- Prompting, retrieval, and generation choices must be explained with respect to quality, latency, and grounding.
- Teams must report at least one quantitative and one qualitative evaluation of outputs.
- The system should remain focused on retrieval quality and grounded response generation rather than large agentic workflows.

5. Evaluation Criteria

Dimension	Weight	Description
Core Objectives	45%	Quality of retrieval, grounding of generated answers, usefulness of summaries, citation clarity, and overall value of the retrieval-augmented assistant.
Code Quality	20%	Clarity of implementation, pipeline structure, retrieval/generation integration, and reproducibility.
Documentation	15%	Knowledge-base documentation, system explanation, evaluation methodology, and setup instructions.
Product Ideation	10%	Usefulness and clarity of the final document-copilot experience for the chosen domain.
Presentation	10%	Demo quality, explanation of results, and ability to discuss trade-offs and failure cases.

6. Deliverables

- Reproducible source code repository
- Prepared knowledge base and ingestion pipeline
- Demo UI or analysis notebook
- Short evaluation report with sample queries, generated answers, and observations



IITISoC Problem Statement

Science & Technology Council, IIT Indore

Inventory Planning Assistant

Domain: AI / ML
Difficulty: Beginner
Team Size: 2–4 members

1. Background

Small businesses and operations teams need to decide not just what demand will be, but what to buy, when to reorder, how much safety stock to keep, and how to respond when uncertainty is high. A raw demand forecast is useful, but the real product value comes from turning that forecast into operational decisions under real constraints such as storage limits, supplier delays, or limited budgets.

This problem focuses on building a lightweight inventory-planning assistant that combines probabilistic forecasting with actionable decision support. The scope remains accessible to beginner teams, but the final product should feel like a usable planning tool rather than a modeling dashboard alone.

2. Problem Statement

Core Objective: Design and implement an inventory-planning assistant for a simple business scenario, such as a store, cafeteria, warehouse, or pharmacy. The system must forecast future demand, estimate uncertainty, and convert those predictions into planning actions such as reorder timing, purchase quantities, stock-risk alerts, or safety-stock suggestions under practical operating constraints.

3. Scope of Work

3.1 Problem Selection and Data

- Use an appropriate time-series or tabular demand dataset, such as store sales, product movement, food demand, or order-volume records.
- Prepare the dataset carefully and explain feature selection, seasonality assumptions, and possible sources of noise.
- Define the downstream planning task clearly, for example reorder decisions, stockout pre-

vention, surplus-risk reduction, or low-budget procurement planning.

3.2 Probabilistic Modeling

- Implement one or more Bayesian or probabilistic forecasting approaches such as Bayesian regression, Gaussian processes, Bayesian structural time-series, or uncertainty-aware ensemble baselines.
- Explain priors, likelihoods, posterior reasoning, or approximation methods where relevant.
- Compare the probabilistic approach against at least one simpler baseline forecast.

3.3 Interpretation and Decision Support

- Build a simple product or dashboard that presents forecasts together with uncertainty bands or confidence intervals.
- Translate predictions into planning outputs such as reorder suggestions, stockout warnings, purchase quantities, low-confidence alerts, or recommended safety stock.
- Add at least one planning layer beyond static prediction, such as what-if simulation, budget-aware reorder recommendations, supplier lead-time handling, or policy-based alerting.
- Highlight where the model is confident, uncertain, or likely to fail.

4. Technical Constraints

- The method must remain mathematically interpretable and well documented.
- Evaluation must include predictive performance and uncertainty-aware analysis.
- The final system must include a concrete planning interface or decision-support surface, not only offline forecasting plots.
- The final system should be runnable end-to-end without hidden manual steps.

5. Evaluation Criteria

Dimension	Weight	Description
Core Objectives	45%	Forecast quality, usefulness of uncertainty estimates, quality of planning recommendations, and value of the final inventory-assistant workflow.
Code Quality	20%	Clean implementation, reproducible experiments, baseline comparisons, and well-structured data/model pipelines.
Documentation	15%	Clarity of assumptions, mathematical reasoning, evaluation methodology, and setup instructions.
Product Ideation	10%	Practical usefulness of the inventory-planning interface and how well uncertainty is surfaced to users.
Presentation	10%	Quality of demo, explanation of model behavior, and clarity in discussing uncertainty and limitations.

6. Deliverables

- Reproducible source code repository
- Processed dataset and documented pipeline
- Interactive planning dashboard or notebook
- Final report summarizing modeling choices, results, and limitations

Intermediate



IITISoC Problem Statement

Science & Technology Council, IIT Indore

Graphitti: Graph-Native Web Intelligence

Domain: AI / ML
Difficulty: Intermediate
Team Size: 4–6 members

1. Background

Most AI systems interact with the web through flat keyword search or stateless RAG over document chunks - with no awareness of the relational structure connecting pages, entities, claims, and data across sites. Real-world queries require reasoning across multiple hops, domains, and versions. This problem asks teams to build a system that crawls the open web on demand, constructs a live queryable knowledge graph via a custom NLP pipeline, and places a multi-agent reasoning and retrieval layer on top - treating the web as a structured graph, not a bag of documents.

2. Problem Statement

Core Objective: Design and implement a graph-based ML system that, given a user query about any public website or domain, crawls and ingests relevant web content into a live heterogeneous knowledge graph, routes the query to the most appropriate retrieval strategy, and answers complex multi-hop questions grounded in the constructed graph with cited provenance.

The system must go substantially beyond retrieval-augmented chat. Teams are expected to design a graph construction pipeline driven by a custom NLP stack, a multi-strategy retrieval engine with comparative benchmarking, and a genuine agentic orchestrator capable of dynamic re-planning. An illustrative direction: a co-pilot over developer documentation portals that answers cross-version queries - reasoning over a version-aware knowledge graph, surfacing contradictions across pages rather than silently merging them.

3. Scope of Work

3.1 Live Crawling and Augmented Dataset Construction

Crawl public pages on demand; follow links to configurable depth; extract structured content from DOM and metadata. Scraping APIs and crawler agents for cleaning are permitted. Run

a custom NLP pipeline - entity recognition, relation extraction, coreference resolution, semantic chunking, and claim extraction with source attribution. Construct a typed heterogeneous knowledge graph; node and edge types are a team design decision. Assign confidence scores, source attribution, and temporal metadata to every edge. Publish the augmented dataset as a citable, versioned artifact - an original contribution.

3.2 Knowledge Graph Schema and Consistency Agent

Design a typed graph schema; run a consistency agent continuously during ingestion. Detect conflicting claims, broken references, and schema violations; store conflicts as contested edges visible to downstream agents - not silently resolved. Design your own conflict representation, resolution protocol, and rollback mechanism.

3.3 NLP Pipeline: Web Content Graph Triples

Semantic chunking with boundary detection - not fixed-window splitting. Named entity recognition, entity linking, relation extraction (typed subject-predicate-object triples), and coreference resolution across pages within a crawl session. Claim extraction with source attribution; context-scoped labeling where applicable. Pipeline evaluated independently on a manually labeled held-out set.

3.4 Query Understanding and Routing

Classify incoming natural-language queries by intent before dispatching retrieval. Routing must be transparent, logged, and ablated - demonstrate it improves over any single fixed strategy. Explore: intent classification, query rephrasing, multi-hop decomposition - design choices are yours.

3.5 Multi-Strategy Retrieval Engine

Implement and benchmark at least three distinct retrieval strategies - e.g. dense vector, sparse/BM25, graph traversal, hybrid fusion, entity-centric (team's choice). Each strategy benchmarked independently: precision, recall, and latency on a labeled test set. Final system must show routing outperforms any single strategy in isolation.

3.6 Multi-Agent Orchestration

Multi-agent architecture with dynamic re-planning when agents return low-confidence results or hit dead ends. Shared typed working memory (scratchpad); orchestrator guarantees termination within a configurable latency ceiling. Functional coverage required: crawling, NLP execution, retrieval selection and execution, answer synthesis with provenance, faithfulness checking, and background graph integrity monitoring. Agent decomposition is a design decision.

4. Technical Constraints

- NLP pipeline and graph construction must be built by the team; scraping APIs permitted.
- Live graph visualization required in the demo.
- All retrieval strategies must be ablated - report individual performance before combining; justify why each strategy was chosen.
- Every design decision (graph schema, agent decomposition, retrieval mix) must be justified with measured impact, not intuition.
- Graph spin-up time (first crawl first successful query) is a primary reported metric.

- Fully reproducible via a documented setup process; open-source tools preferred.

5. Evaluation Criteria

Dimension	Weight	Description
Core Objectives	45%	Successful realization of the graph-native web intelligence pipeline, graph correctness, crawl-to-query spin-up speed, retrieval ablation quality, orchestration re-planning, provenance-grounded answering, and latency percentile reporting.
Code Quality	20%	Modularity, reproducibility, API structure, error handling, separation of concerns, and clean integration between crawling, NLP, graph construction, retrieval, and orchestration layers.
Documentation	15%	Architecture diagrams, benchmark methodology, graph schema design rationale, agent workflow documentation, README completeness, and reproducibility details.
Product Ideation	10%	Practical usefulness of the chosen product surface, clarity of the query workflow, and quality of the graph-backed interaction design.
Presentation	10%	Live demo quality, graph visualization during ingestion, explanation of system trade-offs, and overall clarity of the final presentation.

6. Bonus Extensions

- **Crawl-to-Query Speed Record:** Crawl-to-query in under 60 seconds on a live unseen domain, demonstrated during evaluation.
- **Contested Claim Surfacing:** System detects and explicitly presents contradictory claims from different source pages, with source attribution, rather than silently merging or suppressing them.
- **Browser Extension or Site Copilot:** A functional browser extension that activates on any page, injects the copilot interface, triggers on-demand crawling of the current site, and surfaces query and auto-fill capabilities inline.
- **Continuous Live Updates:** Live crawler agent for continuous KG updates with staleness metrics and delta-only graph patching.
- **Latency Optimization:** Agent-to-agent latency optimization with a measured per-boundary latency breakdown.

7. Deliverables

- **Source Code Repository:** Public source code repository with documented, reproducible setup.
- **Augmented Dataset Artifact:** Versioned, shareable NLP-extracted KG triples.
- **Working Product Surface:** Web app or API demonstrating all query types.
- **Retrieval Ablation Report:** Ablation documentation evaluated with a manually labeled test set.
- **Benchmark Report:** Structured metrics covering spin-up time, P50/P90/P95/P99 latency, throughput, and hardware specifications.
- **Technical Research Report:** Deep architectural analysis covering the NLP pipeline, graph schema design, agent structure, results, and limitations.

- **Media Framework:** Live graph visualization layout during demo alongside a functional project video.

8. Recommended Tech Stack

- **Languages & Storage:** Python, Neo4j / NetworkX
- **NLP & Search Frameworks:** spaCy, FAISS, Hugging Face Transformers
- **Orchestration & Scraping:** Playwright / Scrapy, LangChain, LangGraph



IITISoC Problem Statement

Science & Technology Council, IIT Indore

SPECTRUM

Domain: AI / ML
Difficulty: Intermediate
Team Size: 4–6 members

1. Background

While fine-tuning Large Language Models (LLMs) is standard practice, the exact behavioral, memory, and latency impacts of combining multiple optimization techniques remain poorly understood. Most existing benchmarks evaluate Parameter-Efficient Fine-Tuning (PEFT), quantization, pruning, distillation, and alignment in complete isolation, ignoring their compound interactions.

This problem statement requires teams to close this empirical gap. Teams will conduct rigorous, systematic research mapping how these methodologies interact when applied sequentially or concurrently, culminating in a highly optimized, compressed chatbot application.

2. Problem Statement

Core Objective: Select one open-weight LLM family from Hugging Face (e.g., Qwen2.5, Llama 3, Mistral, Gemma), analyze its underlying architecture, and evaluate the full spectrum of fine-tuning and compression configurations.

The primary research mandate is to compress a large variant of the chosen family (e.g., Qwen2.5-14B) down to the size of its official smaller sibling (e.g., Qwen2.5-7B or 3B) using pruning and knowledge distillation. Teams will then evaluate this custom-compressed model directly against the native open-source small model to answer a core empirical question: *Does a compressed large model outperform a natively trained small model of matched size?*

3. Scope of Work

3.1 Architecture Study & Baseline Profiling

Commit to a single model family and analyze its foundational parameters before initiating any training workflows:

- Profile attention grouping, layer sensitivity, activation magnitudes, and gradient norms

across layers.

- Identify and document which components (attention projections, FFN gates, embeddings) are most or least receptive to specific optimization techniques.
- Establish a definitive baseline benchmark score across all core capabilities prior to model modification.

3.2 Fine-Tuning Execution

- Run full fine-tuning to establish an absolute performance ceiling baseline, ablating learning rates, batch sizes, precision, and training step durations.
- Implement varied PEFT methods (LoRA, QLoRA, DoRA, LoRA+, AdaLoRA, LoftQ, prefix/prompt tuning, IA3) and justify selection choices using findings from the architecture study.
- Record rank, layer-targeting configurations, and hyperparameters with empirical measurements.
- Track catastrophic forgetting curves using a general capability probe at every checkpoint interval.

3.3 Model Compression & Knowledge Distillation

- Apply pruning strategies (unstructured, structured attention heads, FFN channels) and multi-bit post-training quantization (PTQ) or quantization-aware training (QAT).
- Investigate order-of-operation execution tracks (e.g., prune-then-tune vs. tune-then-prune vs. quantize-tune-prune) at matched final model scales.
- Deploy multi-signal knowledge distillation from the larger teacher model to reclaim lost capabilities, exploring output logits (KL divergence, JSD), intermediate hidden states (MSE, CKA alignment), and token- or sequence-level objectives.

3.4 Preference Optimization Alignment

- Apply post-SFT alignment algorithms such as DPO, ORPO, or GRPO.
- Graph and quantify the specific tension between base model capabilities and downstream alignment objectives.
- Directly benchmark a minimum of two distinct alignment objectives to justify the final model behavior.

3.5 Comprehensive Evaluation & Ablation Matrix

- Execute a unified evaluation suite spanning code correctness, mathematical reasoning, and general knowledge retention at every training milestone.
- Construct a detailed interaction matrix logging benchmark scores, VRAM allocation, and processing latency for every tested permutation of PEFT, compression, alignment, and distillation.
- Report median and P95 inference latency under uniform batch sizes, stating exact hardware configurations.

4. Sample Product Directions

Teams must deploy their final optimized, compressed model variant inside a functional, user-facing application surface. Target deployment configurations include:

- **General-Purpose Assistant Interface:** A clean, web-based chat surface showcasing fluid

text generation, low execution latency, and high context retention.

- **Edge-Deployed Copilot:** A lightweight chatbot tailored for local deployment on consumer-grade hardware, highlighting optimized memory footprint reductions.
- **Domain-Specific Reasoning Agent:** A structured conversational application emphasizing specialized knowledge retention (e.g., math or coding) despite high compression ratios.

5. Technical Constraints

- **Model Consistency:** All experimentation tracks must use the exact same base model checkpoint. Changing base models during the evaluation invalidates the interaction matrix.
- **Evaluation Integrity:** Every tested configuration must run the full evaluation suite; partial or selective benchmarking tables are unacceptable.
- **Compute Limits:** The final compressed model variant must execute inference locally on a single 16/24 GB GPU. Any larger compute environments require a cloud verification script.
- **Reproducibility:** Training configurations, random seeds, and specific dataset splits must be tightly version-controlled.

6. Evaluation Criteria

Dimension	Weight	Description
Ablation Study Quality	40%	Interaction matrix completeness; rigorous control parameters; mechanistic explanations of results; full workflow reproducibility.
Architecture Understanding	15%	Depth of baseline analysis; quality of layer sensitivity profiling; evidence of architecture-informed technical choices.
Technique Coverage	15%	Functional depth across PEFT, pruning, quantization, alignment, and distillation implementations.
Code Quality	10%	Pipeline modularity, systematic configuration management, explicit logging, and reproduction clarity.
Final Model and Chatbot	10%	Performance metrics of the compressed model; overall responsiveness and usability of the chatbot product surface.
Presentation and Report	10%	Analytical clarity of the technical report; layout quality of ablation tables; transparent documentation of negative results.

7. Bonus Extensions

- **Training Optimizations:** Implement and measure throughput enhancements via gradient checkpointing, mixed-precision training, FlashAttention, or compilation utilities (‘torch.compile’).
- **Inference Speedups:** Integrate advanced inference optimizations including speculative decoding, continuous batching, custom KV cache tuning, or structural GGUF/GPTQ quantization exports.
- **Scale Verification:** Replicate pivotal ablation matrix findings on an even larger model scale to determine which optimization rules hold true as parameters grow.
- **RLHF Explorations:** Implement online RLHF variants (PPO, ReMax) and directly

benchmark their results against offline preference optimization methods (DPO).

8. Deliverables

- **Codebase Repository:** A public repository containing the unified training pipeline, automated evaluation suite, and explicit reproduction scripts for all configurations.
- **Architecture Report:** Detailed analytical documentation of layer sensitivities, activation profiles, and the initial baseline telemetry.
- **Interaction Matrix:** A master ablation data table explicitly cross-referencing benchmark scores, memory usage, and latency metrics across all method combinations.
- **Capability Evaluations:** Documentation mapping forgetting curves across all incremental check-pointing intervals.
- **Head-to-Head Comparison:** A definitive empirical analysis matching the custom-compressed model directly against its native open-source sibling.
- **Chatbot Application:** A functional general-purpose conversational interface powered directly by the optimized, compressed model, utilizing the recommended tech stack (Python, PyTorch, Hugging Face Transformers/PEFT/TRL, bitsandbytes, vLLM).



IITISoC Problem Statement

Science & Technology Council, IIT Indore

VeriTrace AI: Multimodal Reasoning Auditor

Domain: AI / ML
Difficulty: Intermediate
Team Size: 4–6 members

1. Background

Multimodal Large Language Models (MLLMs) like LLaVA and Qwen2-VL are increasingly utilized across fields such as digital forensics, media analysis, education, and intelligent search. However, establishing user trust remains a challenge because these systems hide their internal reasoning traces, exposing only final answers.

In multimodal environments, models frequently rely on incorrect image regions, ignore vital video frames, over-depend on prompt phrasing, or hallucinate visual details entirely. As AI-generated media grows more pervasive, interpretability and mechanistic forensic auditing of multimodal AI reasoning have emerged as critical challenges.

2. Problem Statement

Core Objective: Design and implement an interactive platform that audits, visualizes, and explains the internal reasoning behavior of open-source MLLMs.

The platform must expose hidden interpretability signals to clarify why a model reached a specific conclusion and evaluate the mathematical stability of that reasoning path. Teams must develop an end-to-end workflow integrating multimodal inference pipelines, explainability frameworks, visual attribution checking, and an interactive dashboard interface.

3. Scope of Work

3.1 Core Ingestion & Interpretability Requirements

Teams must develop an evaluation pipeline where users upload media, input natural-language questions, and review verified model outputs accompanied by clear spatial attribution data.

- Integrate at least one open-source vision-language model (e.g., LLaVA, Qwen2-VL). Smaller architectures (e.g., MDETR, LXMERT) or quantized variants (4-bit/8-bit precision) are

acceptable if local VRAM is a constraint.

- Generate visual question answering (VQA) and structural scene understanding outputs.
- Provide granular interpretability metrics including attention maps, token-to-region highlighting, and comparative views across prompt variations.
- Validate the explainability architecture against quantitative criteria including faithfulness, noise sensitivity, saliency consistency, and structural robustness under intentional input perturbations.
- Ensure all saliency outputs pass fundamental sanity checks, demonstrating sensitivity to both model parameters and underlying data distributions.

3.2 Advanced Auditing & Counterfactual Extensions

Extend the base analytical architecture into a comprehensive system tracking dynamic behavioral drift.

- Expand media support to process short video files and audio clips alongside standard image inputs.
- Implement advanced attribution techniques such as integrated gradients, feature evolution tracking, layer-wise propagation, or representation similarity analysis.
- Construct a dynamic counterfactual sandbox allowing users to mask image regions, drop specific video frames, or alter prompt phrasing to observe real-time variations in outputs.
- Build automated diagnostic alerts, including frame-level relevance scores, response-change indicators, and heuristic hallucination risk flags based on metrics like attention entropy.

4. Technical Constraints

- **Open-Source Mandate:** Rely exclusively on open-source frameworks, models, libraries, and datasets. Paid APIs or proprietary cloud infrastructure services are prohibited.
- **Hardware Accessibility:** Prioritize model selections and quantization states capable of executing inference smoothly on consumer-grade GPUs or open cloud notebook layers.
- **Architectural Modularity:** Design an extensible codebase featuring decoupled, modular pipeline stages so that new feature attribution backends can be added incrementally without rewriting core services.

5. Evaluation Criteria

Dimension	Weight	Description
PS Objectives	45%	Core system goals: robustness of multimodal ingestion, VQA and scene-understanding quality, correctness and usefulness of saliency and attribution outputs, strength of counterfactual and audit workflows, quantitative interpretability validation, and a working analyst-facing interface.
Code Quality	20%	Clean, modular, well-documented code with separation between agents, multimodal pipelines, analysis logic, and output generation.
Documentation	15%	README, architecture diagrams, agent design rationale, evaluation methodology, interpretability workflow guide, limitations, and reproducibility.
Product Ideation	10%	Clarity and usefulness of the interpretability platform concept, quality and explainability of generated analyses, and analyst-readiness of outputs.
Presentation	10%	Demo quality, walkthrough of agent collaboration, example audit cases, and real-world use case demonstration.

6. Bonus Extensions

- **Cross-Modal Attention Analysis:** Incorporate audio streams as an independent input modality to evaluate cross-modal synchronization and joint attention configurations.
- **Multi-Model Comparative Audits:** Integrate a side-by-side evaluation layout matching two different open-source MLLMs against the exact same interpretability tasks.
- **Hallucination Stress Testing:** Set up benchmark evaluation suites intentionally designed to trigger, track, and score visual hallucination frequencies.
- **Long-Form Summaries:** Support complex temporal reasoning and generate unified interpretability overviews across extended video files.
- **Automated Compliance Reports:** Construct an automated utility that compiles audit runs into shareable, print-ready PDF reports with embedded saliency figures.

7. Deliverables

- **Source Repository:** Clean, version-controlled repository housing the inference pipeline, interpretability backend, and automated testing tools.
- **Setup Documentation:** Explicit environment installation guides, required dependency lists, and reproduction scripts.
- **Technical Report:** Detailed document explaining the selected interpretability math, metric evaluation systems, and overall platform architecture.
- **Frontend Dashboard:** A functional, localized web UI built using standard visualization stacks (e.g., React, Next.js, FastAPI, Captum, SHAP) displaying uploaded media alongside real-time saliency overlays.
- **Media Artifacts:** A project presentation slide deck alongside a comprehensive video demonstration showcasing core processing loops and counterfactual stress testing.



IITISoC Problem Statement

Science & Technology Council, IIT Indore

Autonomous Financial Due Diligence Platform

Domain: AI / ML
Difficulty: Intermediate
Team Size: 4–6 members

1. Background

Financial due diligence is a highly critical, manually intensive process in private equity, venture capital, and mergers and acquisitions. Investment analysts routinely spend weeks reviewing hundreds of financial statements, SEC filings, data rooms, and market updates to identify risk factors. This reliance on human review makes the process expensive, slow, and prone to oversight.

Modern large language models matched with multi-agent orchestration can transform this workflow. By synthesizing document processing, cross-period financial reasoning, and automated web information gathering, an intelligent system can compress research timelines from weeks into hours while surfacing deeper, more uniform risk signals.

2. Problem Statement

Core Objective: Design and implement an autonomous financial due diligence platform driven by a coordinated multi-agent team. Given an assortment of public or private target company data, the system must:

- Parse, structure, and normalize erratic unstructured file inputs (PDFs, spreadsheets, SEC records).
- Compute fundamental financial ratios and key performance indicators cleanly without mathematical hallucination.
- Execute programmatic market intelligence lookups using external application interfaces.
- Produce an investment recommendation memo built to professional analyst standards.

3. Scope of Work

3.1 Multi-Agent Architecture & Orchestration

Teams must establish a coordinated agent framework (using tools like LangGraph, CrewAI, AutoGen, or a tailored custom orchestration pipeline) with a minimum of four distinct roles sharing execution logs:

- **Ingestion Agent:** Parses native or scanned inputs and normalizes parameters into an internal data schema.
- **Analysis Agent:** Computes exact financial metrics, charts performance trends, and surfaces raw data balances.
- **Risk Assessment Agent:** Cross-references metrics against benchmarks to flag legal, macro, and asset-level exposures.
- **Market Intelligence Agent:** Drives programmatic API collection loops to grab competitor valuation multiples.
- **Memo Generation Agent:** Collates downstream analytical responses into an executive investment briefing.

3.2 Model Context Protocol (MCP) Tool Integration

The platform must utilize the Model Context Protocol (MCP) as its foundational integration layer to handle execution tools securely.

- Create or connect to at least two functional MCP servers (e.g., pulling live SEC EDGAR files or tracking public financial data feeds via Alpha Vantage or Financial Modeling Prep).
- Expose key systemic actions (such as fetching data arrays or computing specific performance metrics) as clear, declarative, and audit-logged tool signatures.

3.3 Document Processing & Mathematical Execution

- Extract transactional data arrays cleanly from scanned PDFs, organizational spreadsheets, and structured corporate listings.
- Compute core financial expressions automatically, including liquidity metrics, debt profiles, and operational margins across parallel reporting periods.
- Isolate accounting anomalies, sudden margin shrinkages, or atypical asset fluctuations.
- Benchmark historical outputs against centralized sector medians.

3.4 Multi-Dimensional Risk Valuation

The system must generate an analytical risk score across multiple functional areas:

- **Financial:** Tracking capital leverage limits, liquidity thresholds, and active debt covenant violations.
- **Market:** Evaluating addressable market boundaries, competitive friction, and secular industry headwinds.
- **Operational:** Highlighting key-person dependencies, technological gaps, and supply chain exposure vectors.
- **Legal:** Compiling pending litigation hazards and structural regulatory compliance gaps.

Assign clear risk tags (Low, Medium, High) tied directly to verbatim evidence links located within the source documents.

3.5 Product Surface and Reporting Interface

The application layer must be exposed through an accessible interface format (such as a localized web workspace, clean REST API, or command-line system) featuring:

- Streamlined data room file uploads accompanied by automated ingestion state indicators.
- Visual tracking interfaces highlighting current agent steps and tool execution streams.
- Comprehensive briefing downloads that compile a formal executive summary, comparative financial tables, valuation maps, and structured risk assessment matrices into an exchangeable PDF or Word format.

4. Technical Constraints

- **Agent Scale:** Maintain an operational multi-agent framework utilizing at least four unique active roles.
- **MCP Enforcement:** Incorporating the Model Context Protocol is mandatory; at least two unique external tool APIs must be accessed through an MCP interface framework.
- **Computation Rule:** Large language models may evaluate trends, but teams must use deterministic execution layers for quantitative analysis; generative token engines cannot be the sole computation vector for financial arithmetic.
- **Audit Traceability:** Maintain deep, time-stamped system logging throughout execution; hard-coded numbers or untraceable data transformations are prohibited.
- **Infrastructure Limitations:** Build dependencies around open-source runtime systems and accessible, free-tier external APIs that execute cleanly on basic consumer hardware or localized cloud notebook runtime environments.
- **Benchmarking Validation:** Validate system memo clarity and data extraction accuracy against human analyst records across at least three distinct corporate test cases.

5. Evaluation Criteria

Dimension	Weight	Description
Core Objectives	45%	Integration stability of multi-agent frameworks, correct setup of MCP server protocols, mathematical correctness of computed data, risk assessment accuracy, and memo completion.
Code Quality	20%	Modular infrastructure layout, explicit decoupling of data connection handlers, clean separation of agents, and standardized variable validation.
Documentation	15%	Functional environmental onboarding guides, comprehensive structural layouts, explicit design trade-off reasoning, and repeatable testing methods.
Product Ideation	10%	Practical utility of visual dashboard panels, completeness of execution tracking interfaces, and professional presentation of document summaries.
Presentation	10%	Clarity of execution demonstrations, rigorous tracking of agent interactions, and logical breakdowns of corporate use cases.

6. Selection Criteria

Shortlisting considerations include a clear operational understanding of multi-agent execution loops, practical familiarity with corporate balance sheets or regulatory filings, clean integration

approaches for RAG pipelines and tool server systems, and a logical, well-distributed implementation schedule.

7. Bonus Extensions

Advanced engineering groups may choose to implement:

- **Automated DCF Models:** Auto-generating functional discounted cash flow validation spreadsheets from ingestion models.
- **Monte Carlo Simulations:** Incorporating continuous statistical variance modeling across financial projection runs.
- **Contract Review Modules:** Parsing operational contracts to identify internal structural liabilities and red-line hazards.
- **Portfolio Optimization Engines:** Constructing comparative multi-firm rank algorithms to grade target assets across combined valuation scales.
- **ESG Ingestion Backends:** Building a localized compliance extraction agent targeting corporate environmental, social, and governance declarations.

8. Deliverables

- **Codebase Repository:** Clean, version-tracked repository containing modular execution folders, verification cases, and comprehensive README deployment files.
- **Core Engine Blocks:** Functional multi-agent orchestration infrastructure housing the document extraction, data analysis, and multi-dimensional risk engines.
- **Integration Layer:** Active MCP configuration setup linking the agent runtime to at least two distinct external lookup utilities.
- **User Interface:** A localized deployment dashboard displaying active background process tracking and a clean document compilation interface.
- **Technical & Media Documentation:** Comprehensive systems analysis report containing precise flowcharts alongside a brief video walkthrough of the complete end-to-end operation.



IITISoC Problem Statement

Science & Technology Council, IIT Indore

Intelligent Cargo Packing & Spatial Neuro-Optimization

Domain: AI / ML
Difficulty: Intermediate
Team Size: 4–6 members

1. Background

Air-cargo operators rely on Unit Load Devices (ULDs) to move large sets of packages efficiently under tight geometric and operational constraints. The loading plan for a given flight must decide which packages get assigned to which containers, how each package is oriented, and where it is placed, all while respecting container boundaries, weight limits, and shipment-priority requirements. This is a high-impact optimization problem where even small improvements in utilization and planning quality can translate to meaningful operational savings.

At its core, the task is a constrained 3D bin-packing and assignment problem. Classical heuristics can produce feasible layouts, but they often struggle to generalize across diverse package mixes, changing fleet configurations, and competing objectives. This problem invites teams to retain the rigor of the industrial specification while using AI methods, such as learned packing policies, neural scoring models, graph-based spatial representations, or hybrid search systems, to build a stronger decision engine.

2. Problem Statement

Core Objective: Design and implement an AI-driven cargo loading engine for a given flight. Given a fleet of ULDs, a set of packages labeled as Priority or Economy, and a penalty constant for Priority-bearing ULD usage, the system must determine package-to-ULD assignment, placement coordinates, and valid orientations such that:

- (a) No two assigned packages overlap, and every assigned package lies fully inside its chosen ULD.
- (b) The total weight of packages inside any ULD never exceeds that ULD's weight limit.
- (c) All Priority packages are shipped without exception.

(d) Among feasible solutions, the system minimizes total cost:

$$\text{Total Cost} = \sum \text{Cost of Left-Behind Economy Packages} + K \times N_{\text{priority}}$$

where N_{priority} is the number of ULDs containing one or more Priority packages.

3. Scope of Work

3.1 Input Modeling and Environment Formulation

Teams must build a solver-ready environment from structured flight inputs.

- Parse a list of ULDs with identifiers, dimensions, and maximum weight limits.
- Parse a list of packages with identifiers, dimensions, weights, shipment category, and delay cost where applicable.
- Support exactly six valid axis-aligned orientations for each cuboidal package.
- Maintain a geometric state representation capable of checking bounds, support, and collision validity for every placement.

3.2 Intelligent Placement Engine Development

The core implementation should use AI to improve decision quality over naïve packing rules.

- Implement a learned or hybrid decision layer, for example DRL, imitation learning, neural ranking over feasible placements, graph-based scoring, or search guided by learned heuristics.
- Output, for each packed package, a chosen ULD, placement coordinates, and orientation.
- Design state representations such as voxel grids, height maps, free-space trees, or graph encodings that allow the model to reason about residual capacity and future fit.
- Incorporate objectives that reward feasibility, Priority coverage, efficient volume usage, and lower total solution cost.

3.3 Cost-Aware Optimization Architecture

Feasibility is primary; cost is secondary among feasible solutions.

- The system must first guarantee Priority shipment, weight validity, in-bounds placement, and non-overlap.
- Only after satisfying those hard requirements should the optimizer minimize left-behind Economy cost and reduce the number of ULDs carrying Priority packages.
- Teams should justify how the learning objective, search procedure, or ranking model encodes this ordering of priorities.

3.4 Input Scale and Bounding Parameters

The evaluation environment will expose multiple ULDs and a larger incoming package set than the toy examples below. A representative input profile is:

Container ID	Length (cm)	Width (cm)	Height (cm)	Weight Limit (kg)
Sample ULD-1	100	80	80	250
Sample ULD-2	100	80	80	250
Evaluation Maximums	240	320	240	<i>Variable Fleet</i>

Package sets scale dynamically relative to container capacities. Teams should assume heteroge-

neous manifests, mixed package priorities, variable delay costs for Economy items, and larger evaluation instances than the illustrative examples.

3.5 Sample Input Manifest

The following toy example mirrors the structure of the expected evaluation input.

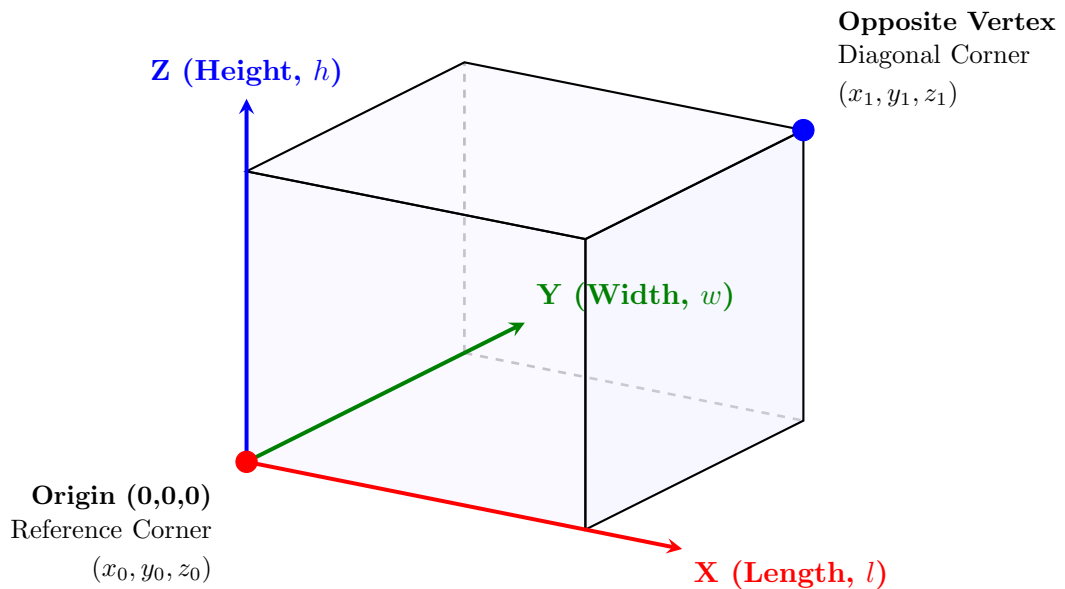
ULD Identifier	Length (cm)	Width (cm)	Height (cm)	Weight Limit (kg)
ULD-1	100	80	80	250
ULD-2	100	80	80	250

Package ID	Length	Width	Height	Weight	Type	Delay Cost
P-1	70	40	50	100	Priority	–
P-2	70	40	50	100	Priority	–
P-3	70	40	50	150	Economy	20
P-4	70	40	50	150	Economy	30

For this toy example, let $K = 40$ for each ULD containing one or more Priority packages.

3.6 Spatial Axis Convention & Mapping Logic

Placements must match a strict Cartesian coordinate convention. Each packed package is represented by two diagonally opposite corners (x_0, y_0, z_0) and (x_1, y_1, z_1) , where length, width, and height correspond to the X, Y, and Z axes respectively. The origin is the front-left-bottom corner of the ULD. The reported reference corner should be the package corner with the smallest z value; if multiple corners tie, choose the one with the smallest y value.



3.7 Product Interface and Reporting Surface

The application layer must be exposed through an accessible interface format featuring:

- Manifest ingestion and validation for ULD and package tables.
- Visual tracking of package assignment, orientation, and step-by-step placement in 3D space.
- Export tooling that emits the exact machine-readable solution format required by evaluation.

3.8 Expected Output Format

The system must emit a final structured assignment file.

- First line: Total-Cost, Total-Packed-Packages, Number-of-Priority-ULDs
- Subsequent lines: Package-ID, ULD-ID, x0, y0, z0, x1, y1, z1
- For packages not loaded into any ULD, use NONE and coordinates -1,-1,-1,-1,-1,-1.
- Output should be consistently formatted and directly parsable by an automated evaluator.

3.9 Sample Output Snippets

Using the toy example above, a feasible solution can place one Priority and one Economy package in each ULD:

- P-1, ULD-1, 5, 0, 0, 45, 70, 50
- P-3, ULD-1, 45, 0, 0, 85, 70, 50

This layout has total cost 80, since both ULDs contain Priority packages and no package is left behind. Another feasible layout can place both Priority packages in one ULD and only one Economy package in the other, leaving one Economy package unpacked; for the toy data above, that yields total cost $60 = 40 + 20$. A layout that packs both Economy packages together with one Priority-heavy ULD may appear cheaper on paper, but it is invalid if the ULD exceeds its 250 kg weight limit.

4. Technical Constraints

- **Physical Feasibility:** Every loaded package must fit entirely within its ULD without overlap.
- **Weight Validity:** The total weight of all packages assigned to a ULD must not exceed that ULD's weight limit.
- **Priority Compliance:** Leaving behind a Priority package is not allowed in a valid solution.
- **AI-Driven Optimization:** Teams should go beyond a trivial one-shot heuristic by incorporating a learned model, learned scoring function, or a hybrid AI-search approach.
- **Evaluation Scale:** Generalization testing will use larger multi-ULD instances with diverse package mixes.
- **Output Standard:** The final solution must match the required structured output format exactly.

5. Evaluation Criteria

Dimension	Weight	Description
PS Objectives	60%	Feasible assignment quality on the challenge instance: all Priority packages shipped, ULD weight limits respected, all packed items fully inside containers, no overlaps, correct output formatting, and lower total cost among feasible solutions. Cost is secondary to validity.
Presentation	20%	Explanatory video review, performance graph tracking logs, and analytical trace demonstrations.
Technical Report	15%	Architecture diagrams, methodology, AI design rationale, feasibility-checking logic, experiments, and limitation analysis.
Code Engineering	5%	Reproducible code, clear structure, readable implementation, and a concise run guide for evaluation.

6. Bonus Extensions

- **Physics-Informed Balance Constraints:** Integrate center-of-gravity (CoG) reward penalty checks to distribute mass distributions smoothly across container floor surfaces.
- **Cooperative Multi-Agent Systems:** Deploy unique coordinate sub-agents managing separate structural zones to optimize layout throughput.
- **Tapered Boundary Adaptation:** Expand environmental constraints to safely track item boundaries along non-cuboidal, tapered structural profiles.

7. Deliverables

- **Source Repository:** Clean, reproducible code containing the packing engine, feasibility checks, model or heuristic components, and instructions to run the system.
- **Technical Report:** A PDF report, up to 10 pages, covering the problem formulation, architecture, modeling choices, experiments, and limitations.
- **Run Guide:** A short how-to document describing exactly how evaluators can execute the solver on a fresh input manifest.
- **Structured Optimization Output:** Exported files providing exact coordinate solutions matching the required evaluation format:
 - *Line 1:* Total-Cost, Total-Packed-Packages, Number-of-Priority-ULDs
 - *Subsequent Lines:* Package-ID, ULD-ID, x0, y0, z0, x1, y1, z1
 - *Unpacked Packages:* Package-ID, NONE, -1, -1, -1, -1, -1, -1

Advanced



IITISoC Problem Statement

Science & Technology Council, IIT Indore

RL-Based Personalized Chess Assistant

Domain: AI / ML
Difficulty: Advanced
Team Size: 4–6 members

1. Background

Traditional chess engines excel at calculating absolute mathematical move sequences, but they are not designed to tutor human players intuitively. Standard analysis toolkits flag an inaccuracy or a blunder without explaining the underlying positional concept, missed strategic objectives, or actionable corrective guidance.

This project bridges reinforcement learning (RL) engine architectures and interactive educational application design. Teams will build a localized chess assistant capable of advanced gameplay evaluation, natural-language strategic breakdowns, and personalized adaptive feedback tailored directly to an individual's evolutionary style.

2. Problem Statement

Core Objective: Design and implement an interactive chess assistant powered by a custom reinforcement learning engine. Given user game data, the platform must:

- Model a comprehensive chess environment to evaluate full game lifecycles.
- Optimize neural policy and value parameters via self-play networks to aim for Grandmaster-level strength.
- Convert complex positional evaluation metrics into plain-language strategic lessons.
- Deploy an incremental tracking framework that adapts engine behavior and study themes based on repeating tactical vulnerabilities.

3. Scope of Work

3.1 Chess Environment and NNUE Engine Design

Teams must establish a secure state handler to coordinate baseline moves, rules, and network evaluations.

- Implement an accurate move verification loop accounting for checks, promotions, castling, en passant, and draw conditions.
- Categorize discrete game states across opening, middlegame, and endgame intervals.
- Structure the engine evaluation framework using a Neural Network Efficiently Updatable (NNUE) architecture instead of raw static material scoring.
- Secure complete Universal Chess Interface (UCI) protocol compliance for the final engine executable.

3.2 Reinforcement Learning Engine Optimization

The core task focuses on advancing model decision-making pipelines beyond initial training boundaries.

- Formulate self-play or engine-guided reinforcement training iterations exploring policy gradients, actor-critic frameworks, PPO, or AlphaZero-style optimization structures.
- Drive long-term positional planning and calculation limits up to Grandmaster-level benchmarks.
- Track tactical strength improvements across all operational phases using systematic bot matches, with Elo evaluated against Stockfish under uniform 30s+1s time controls.

3.3 Game Evaluation & Analysis Tracking

Participants may build game evaluation and analysis tooling to monitor systemic model choices and compute gameplay correctness across a unified dashboard.

- Detect tactical blunders, positional trade calculations, and structural inaccuracies.
- Participants may map relative aggression levels, defensive resourcefulness, and endgame conversion rates across distinct computational configurations for a better accuracy.

3.4 Natural Language Explanation Layer

Transform numeric evaluation data into accessible strategic commentary via language model text synthesis.

- Convert variations in engine score paths into human-readable tactical retrospectives.
- Deconstruct why an original move compromised safety or structure, and outline better alternative plans.
- Verify that textual output remains strictly grounded in engine data matrices to eliminate chess reasoning hallucinations.

3.5 Streaming Personalization Engine

The platform must implement active tracking vectors to log, analyze, and correct user vulnerabilities across historical sessions.

- Maintain a persistent storage profile mapping player game records and categorized mistakes.
- Utilize streaming or incremental machine learning loops to isolate repeated thematic flaws (e.g., poor king safety, hanging pieces, missed tactics, compromised pawn structures).
- Automatically adjust engine interaction sliders (difficulty, aggression, risk tolerance, positional vs. tactical bias) and recommend targeted study modules matched to user histories.

4. Technical Constraints

- **RL Integrity:** The final engine must actively utilize custom reinforcement learning optimizations; serving as a direct pipeline wrapper around existing strong engines like Stockfish or Leela Chess Zero is strictly prohibited.
- **Architectural Baseline:** Implement an NNUE framework for core evaluation tasks; reliance on hard-coded static heuristic position scores is disallowed.
- **API Restrictions:** LLM integrations are permitted for narrative translation and explanation synthesis, but they must not function as the core move generation or tactical decision matrix.
- **Verification Metrics:** Validate engine Elo performance changes programmatically using automated match scripts against Stockfish at explicit 30s+1s time controls.
- **Compute Boundaries:** Code structures must operate efficiently on consumer hardware tiers, utilizing open-source machine learning and framework toolkits.

5. Evaluation Criteria

Dimension	Weight	Description
Core Objectives	45%	Successful realization of the custom RL optimization loop, attaining high-level play metrics, robust environment handling, precise LLM explanation layers, streaming personalization, and interface completeness.
Code Quality	20%	Clear component decoupling separating environmental logic, policy network training, inference pipelines, explanation handlers, and frontend states.
Documentation	15%	Exhaustive setup guides, explicit training configuration logs, reproducible baseline telemetry, and robust Elo benchmarking logs.
Product Ideation	10%	Practical utility for human learners, clarity of personalized feedback loops, and meaningful responsiveness of operational sliders like aggression or difficulty.
Presentation	10%	Walkthrough performance, validation via concrete chess examples, clear model comparisons, and report professionalism.

6. Bonus Extensions

Advanced development groups may choose to implement:

- **AlphaZero MCTS:** Integrating Monte Carlo Tree Search optimization layers into the move selection pipeline.
- **Endgame Tablebase Hook:** Coupling the engine with specialized external endgame tablebases to secure absolute late-game evaluation accuracy.
- **Live Matchmaking Workspaces:** Developing a fluid, real-time client application interface enabling direct live gameplay against variable engine profiles.
- **Opening Repertoire Deep-Dives:** Building diagnostic features that map player opening trends against standard theoretical classification tables to highlight initial development risks.

7. Deliverables

- **Codebase Repository:** A clean, version-controlled public repository containing full engine files, training configurations, explanation layers, and setup deployment instructions.
- **UCI Engine Executable:** A functional, compiled engine binary utilizing the optimized NNUE policy network weights.
- **Product Workspace Interface:** An accessible deployment surface (web platform, API layer, or desktop interface) featuring multi-parameter adjustment sliders and detailed PGN upload review capabilities.
- **Technical Benchmark Documentation:** A research-driven engineering report compiling model network parameters, RL training methodology logs, inference speed profiles, and comprehensive evaluation charts alongside an explanatory product video.



IITISoC Problem Statement

Science & Technology Council, IIT Indore

SPARK: Accelerated Image Generation

Domain: AI / ML
Difficulty: Advanced
Team Size: 4–6 members

1. Background

Modern generative image models produce state-of-the-art visual outputs, but deploying them within production-tier software surfaces introduces major systems bottlenecks. Standard diffusion and transformer pipelines suffer from steep execution latency, slow container cold starts, large operational VRAM footprints, and rapid degradation under concurrent multi-user request traffic.

This problem statement shifts focus from pure visual aesthetics to end-to-end inference engineering. Teams must design a production-grade image generation backend optimized for rapid inference, minimized memory utilization, and reliable horizontal serving elasticity under structural hardware constraints.

2. Problem Statement

Core Objective: Design, optimize, and deploy a scalable image generation application featuring a high-performance custom inference engine. The complete end-to-end system must:

- (a) Execute all image generation pipelines locally within a strict 16 GB VRAM envelope.
- (b) Bring image generation latency down so that P99 latency, excluding cold starts, stays under 15 seconds per output.
- (c) Systematically mitigate container and weights initialization cold-start overhead.
- (d) Provide rigorous load testing metrics mapping system throughput and latency distribution profiles.

Teams must deploy and optimize the core architecture directly rather than serving outputs via external commercial generative API endpoints.

3. Scope of Work

3.1 Model Selection & Structural Justification

Explore, evaluate, and benchmark foundational generative vision configurations to establish an optimized base engine layer.

- Analyze and contrast a minimum of two open model families or variant scales.
- Investigate modern alternatives beyond standard diffusion, exploring transformer-based text-to-image variations.
- Present a clear architectural justification explaining final model selections.

3.2 End-to-End Inference Optimization

Implement, ablate, and document a variety of computational execution strategies to decrease latency while retaining image quality:

- **Precision & Quantization:** Deploy lower-precision inference modes (FP16, BF16) or structured graph quantization tracking (INT8).
- **Compilation & Execution Graphs:** Integrate model compilation tools via native compilation wrappers ('torch.compile'), AOT Inductor, or ONNX graphs.
- **Hardware Acceleration:** Build and scale accelerated runtimes utilizing TensorRT engines, customized CUDA Graphs, or low-level CUDA kernels.
- Document all optimization attempts explicitly, logging successes alongside technical failure cases.

3.3 Cold-Start Optimization Strategy

Minimize the initialization delta separating container deployment from the first successful image generation output.

- Trace and measure model weight loading latency, container startup times, and GPU allocation overhead.
- Implement performance-recovery methods such as local weights caching, worker preloading strategies, compiled graph storage, or warm worker pools.
- Report precise cold-start measurements across pre- and post-optimization configurations.

3.4 Agentic & Auxiliary System Integration

Incorporate a lightweight orchestration framework around the generation engine to handle user inputs intelligently.

- Deploy language agents to manage automated prompt rewriting, safety filters, context enhancement, or resolution parameter routing.
- For language optimization elements, options include fast frameworks like vLLM, TensorRT-LLM, or accessible high-throughput endpoint infrastructure.

3.5 Deployment, & Load Testing

- Package the entire multi-service ecosystem using isolated container instances managed via Docker
- Conduct rigorous multi-user load testing to map backend performance across variable concurrency steps.
- Report definitive system metrics: VRAM allocation curves, processing throughput, opera-

tional failure rates, request queue latencies, and generation timing across P50, P90, P95, and P99 intervals.

4. Sample Product Directions

Teams are highly encouraged to invent their own original application context leveraging the underlying inference framework. Illustrative archetypes include:

- **End-to-End Image Editing Studio:** A complete image editing and manipulation system supporting guided transformations, localized edits, and rapid iteration on user-provided assets.
- **Instant Social Photo Generator:** A fast photo-generation system with strong character consistency across outputs, tailored for repeated use in social media applications.
- **Ad & Poster Maker:** A production-ready visual generation pipeline for ads, posters, and marketing creatives with quick turnaround and scalable rendering.

5. Technical Constraints

- **Memory Boundary:** The primary image generation pipeline must stay securely under a strict 16 GB VRAM limit.
- **Inference Ceiling:** The optimized pipeline must target P99 image generation latency, excluding cold starts, of under 15 seconds.
- **Ownership Mandate:** The core visual generation pipeline must be completely self-hosted; third-party commercial image APIs are strictly prohibited.
- **Framework Priority:** PyTorch must remain the structural foundation framework unless explicit architectural constraints mandate specialized alternatives.
- **Telemetry Transparency:** Every benchmark dashboard, report, or matrix must explicitly state the exact hardware, model weights scale, and concurrent load conditions used.

6. Evaluation Criteria

Dimension	Weight	Description
Core Objectives	45%	Successful realization of an image generation pipeline within the 16 GB VRAM ceiling with P99 latency under 15 seconds excluding cold starts, verified cold-start compression, stable load handling, and execution optimizations (Quantization, Compilation, TensorRT, or CUDA-level tuning).
Code Engineering	20%	Modular infrastructure layout, robust API structures, clear error isolation, standardized environment containerization, and explicit separation between serving logic and optimization code.
Documentation	15%	Thorough setup instructions, clean parameter specs, detailed compilation notes, hardware configurations, full reproduction tracking logs, and a final technical report.
Product Ideation	10%	Practical utility and implementation of the chosen product direction, paired with smooth user flow mappings and auxiliary agent features.
Presentation	10%	Analytical clarity of the video demonstration, methodical layout of trade-off data, and overall professional quality of the technical submission.

7. Bonus Extensions

Highly capable development teams may choose to explore the following advanced system extensions:

- **Self-Hosted Language Model (+5%):** Deploy, serve, and optimize a local LLM prompt-enhancement agent utilizing customized execution backends (vLLM, TensorRT-LLM) without external API dependencies. This bonus applies only if the deployed model is used everywhere in the agentic system. This component operates outside the image engine's 16 GB VRAM ceiling.
- **Serving Optimizations:** Build active dynamic request batching and automated infrastructure autoscaling driven by queue thresholds or active GPU usage telemetry.
- **Advanced System Metrics:** Construct live Prometheus and Grafana monitoring panels plotting processing latency metrics, queue delays, and cost-per-image estimations.
- **A/B Validation Surfaces:** Create an interactive evaluator view running side-by-side A/B tests matching the unoptimized baseline model directly against the accelerated production pipeline.

8. Deliverables

- **Codebase Repository:** A version-controlled repository containing setup scripts, dockerized deployment files, and automated evaluation toolchains.
- **Optimized Engine Endpoint:** A fully functional image generation API service or web application operating within the 16 GB VRAM budget.
- **Performance Optimization Matrix:** A data-driven benchmark report logging generation latency (P50 to P99), system throughput, cold-start profiles, and memory tracking under load.
- **Technical Research Report:** A comprehensive engineering brief explaining the final system layout, model parameters, acceleration choices, and a complete architectural demonstration video.



IITISoC Problem Statement

Science & Technology Council, IIT Indore

Personalized Mental Health Digital Twin using AI

Domain: AI / ML
Difficulty: Advanced
Team Size: 4–6 members

1. Background

Mental health conditions often develop gradually and vary significantly across individuals, making early detection and intervention challenging. Traditional AI systems rely on static labels (e.g., depressed / not depressed), which fail to capture the dynamic and personalized nature of human cognition and behavior.

This project aims to develop a Personalized Mental Health Digital Twin, an AI-driven virtual representation of an individual's mental state that continuously learns from behavioral data, establishes a personal baseline, detects deviations, and predicts future mental health risks.

2. Objectives

The core objectives of the project are to:

- Develop a personalized baseline model and platform (app/web) of user behavior
- Continuously track and model temporal changes in mental state
- Detect early deviations from normal cognitive and behavioral patterns
- Predict future mental health risks using longitudinal data
- Ensure interpretability, privacy, and ethical AI deployment

3. Core Components

3.1 Baseline Behavior Modeling

Learn individual-specific patterns from:

- Text (journals, chats, writing style)
- Interaction data (typing dynamics, activity patterns)
- Optional: speech or wearable signals

Capture:

- Linguistic complexity
- Behavioral consistency
- Cognitive signatures

3.2 Temporal Mental State Modeling

- Represent mental state as a dynamic trajectory over time
- Track gradual transitions instead of discrete labels

Identify:

- Drift in cognition
- Sudden anomalies

3.3 Deviation & Anomaly Detection

Detect early warning signals such as:

- Changes in language structure
- Increased variability in behavior
- Reduced engagement or irregular patterns
- Use personalized thresholds instead of population-level norms

3.4 Predictive Risk Modeling

Forecast potential mental health risks:

- Stress escalation
- Burnout
- Depressive tendencies
- Enable proactive intervention signals

3.5 Explainability & Feedback Layer

Provide interpretable outputs such as:

- "Significant deviation in writing coherence"
- "Increased behavioral irregularity over past week"
- Build user/clinician trust

4. Relevant Research Directions

- Digital biomarkers in mental health (Nature Digital Medicine, Lancet Digital Health)
- Longitudinal modeling of behavioral data
- Personalized AI and digital twin systems
- Human-centered and explainable AI

5. Evaluation Criteria

Dimension	Weight	Description
Core Objectives	45%	Successful realization of a personalized digital twin pipeline, quality of baseline behavior modeling, temporal tracking and deviation detection, predictive risk modeling, explainability, and practical monitoring interface design.
Code Engineering	20%	Modular infrastructure, clean data-processing and modeling pipelines, clear separation between ingestion, temporal analysis, prediction, and interface layers, and robust implementation quality.
Documentation	15%	Thorough setup instructions, data and modeling assumptions, evaluation methodology, ethical and privacy considerations, reproducibility details, and a final technical report.
Product Ideation	10%	Practical utility of the digital twin concept, clarity of the monitoring and intervention workflow, and usefulness of outputs for users or clinicians.
Presentation	10%	Quality of the demonstration, clarity of the system walk-through, explanation of behavioral insights, and professionalism of the overall submission.

6. Key Deliverables

- Functional Digital Twin system prototype
- Personalized baseline modeling module
- Temporal deviation detection system
- Risk prediction model with evaluation metrics
- Explainability dashboard / visualization
- Clean, reproducible GitHub repository
- Final report (research paper style preferred)

7. Expected Outcomes

- A next-generation AI system for personalized mental health monitoring
- Insights into individual behavioral and cognitive patterns
- Foundation for real-world deployment in digital health platforms
- Potential for high-impact research publication or startup development

8. Extensions

- Multimodal digital twin (text + speech + physiological signals)
- Reinforcement learning for adaptive interventions
- Integration with mobile or wearable platforms
- Long-term longitudinal simulation of mental health trajectories



IITISoC Problem Statement

Science & Technology Council, IIT Indore

Pathway-Powered Agentic Gateway

Domain: AI / ML
Difficulty: Advanced
Team Size: 4–6 members

1. Background

Current LLM agent frameworks frequently collapse under production-grade infrastructure requirements. While highly capable in isolated, single-user demonstrations, they suffer from catastrophic state drift, high orchestration overhead, and a lack of resilience when executing parallel tool calls at scale.

Participants are tasked with building the core routing, stream-processing, and telemetry layer that sits between incoming user requests and upstream AI model providers. To resolve existing concurrency and reliability bottlenecks, this architecture mandates the integration of two distinct paradigms:

- **Pathway Streaming Backbone:** Pathway must be utilized to build a stateful, real-time data streaming pipeline. It serves as the single source of truth for the active “World State,” handling context streaming, session updates, and parallel tool-execution logs incrementally and concurrently.
- **Asynchronous Neural Watchdog (BDH):** To prevent agent reasoning loops, hallucinations, or runaway tool calls from impacting user latency, a Baby Dragon Hatchling (BDH) architecture implemented here as an isolated, low-latency asynchronous microservice (e.g., a fine-tuned SLM or a fast, deterministic reasoning-validation script) must act as a “Neural Watchdog.” By sniffing the active Pathway event stream out-of-band, it flags anomalies and logical deadlocks without blocking the primary request-response lifecycle.

The primary deliverable of this Problem Statement is strictly the Infrastructure Layer. However, to validate its real-world performance, teams must deploy a functional, low-overhead end-user application proxy. The chosen product logic should be complex enough to stress-test the pipeline, but simple enough not to distract from the core systems engineering goals.

2. Problem Statement

The core objective of this problem statement is to design and implement a provider-agnostic, low-overhead agentic orchestration gateway that operates as an infrastructure proxy between user queries and AI model backends. Participants must deliver a highly resilient, production-grade system that eliminates the critical bottlenecks of multi-agent state drift, cascade failures, and excessive orchestration lag under concurrent workloads.

2.1 Core Objectives

- **Latency:** Minimizing the “Query-to-Action” delta. The gateway’s internal routing, state serialization, and context injection logic must introduce negligible processing overhead to guarantee a highly responsive, “live” user experience.
- **Throughput:** Maximizing event ingestion capacity. The orchestration layer must effortlessly digest and process a high volume of concurrent user inputs, active agent thought logs, and parallel tool dispatches simultaneously without encountering backpressure or dropping telemetry data.
- **Scalability:** Ensuring linear or sub-linear system performance. As the volume of concurrent sessions and active agent workflows expands, the architecture must efficiently distribute heavy execution workloads across available compute resources without a compounding degradation in response times.
- **Fault Tolerance:** Engineering absolute architectural resilience against common operational disruptions (e.g., worker node crashes, container terminations, network partitions, or upstream API timeouts). The gateway must seamlessly recover from unexpected failures, leveraging reliable state-checkpointing to resume task execution from the exact last successful sub-step without triggering duplicate tool calls, data loss, or forcing full-sequence retries.

2.2 Model, API, and Parameter Empirical Optimization

Teams must conduct systematic empirical research to map out the exact frontier of the speed-to-accuracy trade-off for their agentic workloads. Finding the mathematical sweet spot where orchestration latency drops without degrading agent reasoning quality is a core requirement. The research must explicitly isolate and measure:

- **Provider Profiling:** Benchmark and compare various APIs specifically on function/tool-calling success rates.
- **Hyperparameter Space Exploration:** Quantify how variations in generation parameters such as Temperature, Top-P, and token penalty weights affect the determinism of agent execution loops versus overall generation speeds.
- **Granular Latency Tracking:**
 - *Time to First Token (TTFT):* To measure immediate gateway and system responsiveness.
 - *Total Task Execution Latency:* The cumulative end-to-end time elapsed across sequential tool dispatches.

3. Scope of Work

3.1 Provider-Agnostic Model Abstraction

The gateway must decouple agent state management from specific LLM providers:

- **Unified Adapter Layer:** Implement an Adapter or Factory pattern supporting interchangeable runtime backends (OpenAI, Anthropic, or local VLLM streaming endpoints) via global configuration toggles.
- **Structured Tool-Calling Contract:** Define a strict JSON schema specification for agent tool-dispatches, ensuring variations in provider function-calling syntax are normalized before hitting the Pathway stream.

3.2 State Stream & Watchdog Integration

- **Incremental ETL State updates:** Every user prompt, model thoughts block, tool invocation, and environment response must be treated as an immutable event appended to the Pathway stream.
- **Out-of-Band Auditing:** The BDH watchdog must consume this event stream asynchronously. If it detects a repeating loop (e.g., the same tool called three times with identical parameters without progress), it must flag the anomaly and emit a high-priority “Intervention Alert.”

3.3 Observability Dashboard & User Proxy

- **Client Proxy App:** A responsive UI (e.g., WebSockets-based chat or control interface) demonstrating real-time multi-agent cooperation.
- **Live Telemetry View:** A dedicated developer dashboard rendering live infrastructure performance metrics, including:
 - Gateway processing overhead vs. upstream model provider latency.
 - Event ingestion rate (events per second) through the Pathway pipeline.
 - Real-time state-graph visualization and active BDH watchdog interventions and alerts.

4. Sample Product Directions

To validate the architecture’s efficiency, concurrency handling, and fault-tolerance frameworks, teams must deploy a functional application proxy. The chosen product use-case must serve the sole purpose of highlighting infrastructure performance; over-complicating application-level features will directly hinder progress on the core engineering goals.

- **DOM Manipulation & Web Automation System:** An agentic proxy that ingests a continuous, high-frequency stream of browser Document Object Model (DOM) mutations and accessibility trees to execute deterministic UI actions.
- **Real-Time Fleet Orchestrator:** Agents responding dynamically to a continuous, high-volume stream of live GPS coordinates and engine telemetry data, dispatching physical routing updates and handling worker failovers seamlessly.
- **Dynamic Support Tiering:** An infrastructure gateway that ingests and parses thousands of concurrent user support tickets, system exceptions, or error logs, routing tasks to specialized model agents based on live cluster health.
- **High-Frequency Financial Monitor:** A system monitoring live trading data feeds, performing out-of-band compliance auditing via the watchdog, and routing immediate anomalous alerts to execution agents without degrading the primary data stream.

5. Technical Constraints

- **Mandatory Containerization:** The entire multi-service application, the gateway, the Pathway processing engine, the BDH watchdog service, any mock tool microservices, and

the UI must be fully decoupled and orchestrated using Docker Compose.

- **State Management & Agent Orchestration:** Teams must evaluate the optimal architecture for balancing rapid development against fault tolerance. While Pathway must remain the ultimate source of truth (the ETL layer) for state recovery, teams may choose between two approaches for their agent logic:
 - *Option A (Framework-Driven):* Utilize established agent libraries (e.g., LangGraph, AutoGen, CrewAI). Because these frameworks inherently hold state in local worker memory during reasoning loops, teams must engineer a robust mechanism to continuously flush memory checkpoints to the Pathway stream to minimize data loss if a container crashes mid-task.
 - *Option B (Zero-State Custom Loops):* Build lightweight, custom reasoning loops where context is never held in local memory. All session state is strictly read from and written to the Pathway stream at every step. This achieves true “Zero Leaked State” and immediate fault tolerance, but requires more manual orchestration logic.
- **Local Execution:** The entire demo product must be able to run entirely on a single (or a few) consumer PCs (e.g., a modern multi-core CPU/GPU setup with standard RAM and VRAM). Use of external API providers is allowed.
- **Frontend Responsiveness:** The UI must reflect the “snappy” nature of the backend, rendering streaming responses and agent state changes.

6. Proposal Evaluation & Selection Criteria

1. **Detailed Architectural Topology Diagram:** A clear block diagram demonstrating the data flow from the client application, through the gateway, into the Pathway stream, and out to model providers. Crucially, it must explicitly illustrate the asynchronous, non-blocking connection to the BDH watchdog service.
2. **Scoped Demonstration Blueprint:** Teams must pitch a concrete product use-case (e.g., a real-time IoT fleet distributor or automated multi-source data reconciliation tool). The proposal will be graded on whether the product logic is complex enough to trigger parallel tool execution while remaining tightly scoped to prevent “product feature creep.”
3. **State Management & Recovery Strategy:** A concise technical breakdown of how they intend to capture, persist, and recover the agentic state graph without forcing full-sequence retries when a container is killed.
4. **Concrete Testing & Chaos Engineering Plan:** An explicit methodology for how the team intends to simulate infrastructure failures (e.g., manual script-driven container restarts mid-task) to generate their “Failure Replay” deliverable.

7. Evaluation Criteria

Dimension	Weight	Description
Implementation	30%	Correctness, throughput, and fault tolerance of the containerized multi-service gateway, including efficient state-checkpointing and zero-leaked state execution.
Architecture	25%	Robustness of the Pathway streaming backbone, asynchronous non-blocking BDH watchdog integration, and decoupled provider-agnostic adapter layer.
Product Implementation	20%	Functionality and responsiveness of the client proxy app and live telemetry dashboard, alongside successful live performance and failure replay demonstrations.
Documentation	15%	Clarity of the technical specification document, empirical optimization report, API schemas, and architectural topology diagrams.
Ideation & Novelty	10%	Relevance and scoping of the demonstration blueprint to stress-test infrastructure without feature creep, alongside analytical depth in hyperparameter exploration, chaos engineering methodology, and innovative approaches to BDH out-of-band auditing.

8. Deliverables

1. **Technical Specification Document:** Comprehensive documentation detailing gateway API schemas, state management architectures, and watchdog intervention protocols.
2. **Empirical Optimization & Benchmarking Report:** A technical research report documenting the team's methodology and findings for achieving optimal gateway performance.
3. **Containerized Orchestration Source:** A fully reproducible, multi-container Docker infrastructure uploaded as a GitHub repository, ideally runnable by just adding environment variables and running Docker Compose.
4. **Live Performance Validation Video:** A recorded run demonstrating gateway latency handling concurrent requests, backed by clean telemetry metrics.
5. **Failure Replay Video:** A screen-recorded verification showing a critical worker node or tool container being hard-killed during an active agent task loop, demonstrating immediate state recovery and uninterrupted task completion.



IITISoC Problem Statement

Science & Technology Council, IIT Indore

Building Lightweight Multi-Task AI with MALORA

Domain: AI / ML
Difficulty: Advanced
Team Size: 4–6 members

1. Background

When adapting language models to highly specialized professional domains, systems face severe performance trade-offs. Optimizing a model for quantitative finance might degrade its software engineering capabilities, and vice versa. Integrating a sparse Mixture-of-Experts (MoE) architecture with low-rank adapters allows a system to dynamically route different domain tasks to independent expert modules.

Standard parallel multi-expert methods, however, introduce massive parameter redundancy, particularly in the down-projection matrices where experts learn near-identical baseline features. By executing a targeted architectural restructuring known as MALORA (Mixture of Asymmetric Low-Rank Adaptation), teams can isolate these overlapping representations into a frozen, shared low-rank subspace while widening task-specific up-projection experts. This enables the production of highly specialized, multi-task AI agents optimized for secure, local, air-gapped deployment.

2. Problem Statement

Core Objective: The objective is to design, implement, and validate a secure, Local AI Workspace by applying structural surgery to an open-weight dense model family (e.g., Qwen2.5-7B or Llama-3-8B).

All teams must build the underlying MALORA MoE tensor engine from scratch. However, teams must choose ONE specialized domain track to build their final product surface around. The final deliverable must be an end-to-end local desktop software application featuring an optimized inference server, a track-specific interactive workspace, and a live diagnostic telemetry dashboard tracking token throughput and active expert routing paths.

3. Scope of Work

3.1 Part A: Core Technical Engine

- **Restructuring the Model’s Math:** Adjust the weight matrices in the feed-forward network to be more efficient. Instead of having every expert relearn the exact same basic representations, you will extract these common baseline features into one frozen, shared component. Then, you will allocate more capacity (higher rank) to the specialized experts so they can focus entirely on their unique domains.
- **Custom Gating Router:** Build a parameterized Top-K token gating layer from scratch using pure tensor math, including an explicit auxiliary load-balancing loss term to prevent routing collapse.
- **Live Routing Telemetry:** A real-time monitoring dashboard embedded in the client UI that exposes live data processing speeds (tokens/sec) and a visual representation showing active parameter routing percentages across experts.

3.2 Part B: Domain Tracks

Teams must choose exactly ONE of the following tracks:

- **Track 1: Financial Intelligence Terminal (FinTech):**
 - Experts to Train: Quantitative Strategy, Asset Valuation, Regulatory Compliance, or any other expert you deem necessary.
 - Product UI: A quantitative backtesting workspace and a mathematical scratchpad for step-by-step options pricing/risk matrices.
 - Evaluation Focus: Strict correctness on FinQA, ConvFinQA, SEC filing QA, and earnings-call sentiment analysis.
- **Track 2: Autonomous Software Engineering (Code Gen):**
 - Experts to Train: Algorithmic Logic, Multi-Language Syntax (Python/C++/JS), Debugging/Security.
 - Product UI: An interactive multi-turn coding IDE optimized for continuous programming assistance, featuring syntax highlighting, language auto-detection, and simulated code-execution safety checks.
 - Evaluation Focus: High pass rates on functional coding execution benchmarks (HumanEval, MBPP, MultiPL-E) and algorithmic optimization tasks.
- **Track 3: Scientific & Medical Reasoning:**
 - Experts to Train: Biomedical Literature Parsing, Physics/Chemistry Step-by-Step Reasoning, Data Interpretation.
 - Product UI: A split-screen research assistant that renders complex step-by-step mathematical/chemical derivations and cross-references academic text.
 - Evaluation Focus: High accuracy on PubMedQA, GSM8K, MATH, and standard MMLU science splits.

4. Technical Constraints

- **Deterministic Evaluation Bounds:** All final model checkpoints must be scored through an immutable, local validation harness script. No custom data tokenizers or dynamic regex script updates are allowed.
- **No High-Level Automation Frameworks:** Automated model-merging engines, black-box pipeline packages, and out-of-the-box MoE cloning suites are explicitly banned. Every

tensor operation, router allocation, and system orchestration step must be natively implemented.

- **Data Privacy:** The system must run entirely on consumer-grade local hardware. No paid APIs, proprietary external services, or cloud compute offloading allowed during inference.

5. Evaluation Criteria

Dimension	Weight	Description
Core Objectives	45%	Product functionality, end-to-end responsiveness, track-specific benchmark accuracy, and correctness of the MALORA tensor engine, including routing behavior and low-level implementation quality.
Code Quality	20%	Explicit modular design, type hinting, robust error handling for local execution, and flawless end-to-end setup reproducibility.
Documentation	15%	Clarity of the methodology, architecture explanation, ablation reporting, benchmark setup, and final technical report.
Product Ideation	10%	How well the chosen track is translated into a coherent local product surface and interactive workspace.
Presentation	10%	Demo quality, clarity of expert-routing explanation, and ability to justify technical trade-offs and results.

6. Deliverables

- **Source Code Repository:** Public GitHub repository featuring the core layer transformation scripts, router modules, backend API pipelines, and the track-specific user interface.
- **Verified Model Artifact:** The optimized multi-expert model configuration packaged with reproducible weight files and a clean invocation script.
- **Automated Ablation Study Matrix:** A clean empirical results grid detailing how adjustments to expert quantities, asymmetric rank ratios, and loss balancing coefficients modify track-specific benchmark correctness.
- **Functional Product Surface:** A deployable local application running real-time interactive inference tailored to the chosen track.
- **Final Report & Demo Video:** A technical report explaining the methodology, alongside a demo video showcasing features and workflow. Teams must explicitly declare their chosen track on the title page.